



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

访问者模式（Visitor）

一、意图（Intent）

表示一个作用于某对象结构中的各元素的操作，使你可以在不改变各元素类的前提下定义作用于这些元素的新操作。

翻成嵌入式 C 的表达，更容易理解成：

数据结构保持稳定，而对这批结构执行什么操作，可以在结构外部继续扩展。

它控制的是：

- 元素结构是否稳定
- 新操作应该加在元素内部还是外部
- 遍历与处理职责如何分离
- 新增操作时是否要改原有节点结构

一句话记住它：

节点结构尽量不动，新操作放在外部访问逻辑里追加。

二、动机（Motivation）

访问者模式在纯 MCU 业务代码里不像观察者、状态那样高频，但在下面这类地方非常有价值：

- 配置树 / 设备树遍历
- 协议树 / 报文树处理
- AST / 脚本语法树处理
- 调试导出、统计、校验、生成代码等“多种操作叠加在同一结构上”

最直接的坏味道是：

- 每个节点类型里塞满各种无关操作
- 想增加一种新分析功能，就要修改所有节点处理函数
- 结构本身和操作逻辑强耦合

这在嵌入式工程里尤其麻烦，因为很多“结构”是很稳定的：

- 配置树格式很稳定
- 报文节点格式很稳定
- 设备描述结构很稳定

真正经常变化的，反而是“对它做什么”：

- 导出
- 校验
- 统计
- 打印
- 生成表
- 生成初始化代码

所以访问者模式真正想解决的是：

元素结构不想频繁改，但新操作会持续增加。

三、适用性（Applicability）

适合考虑访问者模式的条件包括：

- 有一组相对稳定的数据结构节点
- 未来会不断增加新的处理操作
- 不希望每次加新操作都改节点定义
- 操作天然依赖“遍历一整棵结构”
- 结构层与处理层需要分离

常见场景：

- 设备树 / 配置树处理
 - 协议树解析后的二次处理
 - 脚本 / 规则树分析
 - 调试打印、校验、统计、生成代码
-

四、结构（Structure）

```
Object Structure
|
+-- Element A
+-- Element B
+-- Element C

Visitor X
Visitor Y
Visitor Z
```

在嵌入式 C 里更常见的落地是：

```
树 / 节点集合
|
+-- 遍历框架
    |
    +-- 访问操作A（打印）
    +-- 访问操作B（校验）
    +-- 访问操作C（导出）
```

也就是说，很多 C 工程不会严格实现 GoF 式“双分派”，
但会实现它的核心意图：

- 节点结构保持稳定
- 新操作以外置访问逻辑增加

五、参与者（Participants）

1. Element（元素）

表示结构中的节点。

在嵌入式里可以是：

- 设备节点
- 配置节点
- 协议字段节点
- 语法树节点

2. Visitor（访问者）

表示一种“要对节点做的操作”。

例如：

- 打印访问者
- 校验访问者

- 统计访问者
- 代码生成访问者

3. Object Structure（对象结构）

保存整组节点并提供遍历入口。

六、协作（Collaborations）

执行过程通常是：

1. 系统拿到一组稳定节点结构
2. 选定某种访问操作
3. 遍历框架按节点顺序访问
4. 访问者对不同节点执行对应逻辑
5. 遍历结束后得到结果

访问者模式的关键不在“有遍历”本身，而在：

■ 新增操作时，不想去改原有节点定义和已有节点代码。

七、效果（Consequences）

收益

1. 新增操作更容易

增加一种新处理方式，主要新增访问逻辑，不必改所有节点结构。

2. 数据结构更稳定

节点职责更聚焦在“表达结构”，而不是承担一大堆无关行为。

3. 适合工具链与分析类任务

例如打印、校验、导出、生成代码，这些操作天然适合外置。

代价

1. 新增节点类型会牵动访问者

访问者模式对“新增操作”友好，但对“新增元素类型”不如前者友好。

2. 在纯 C 中做满配 GoF 形式偏重

很多嵌入式项目会只保留思想，不硬做完整双分派结构。

3. 运行期 MCU 业务里不一定高频

它更常见于配置树、协议树、工具链和生成侧，而不是简单驱动层。

八、实现要点 (Implementation)

1. 先判断你面对的是“稳定结构 + 多操作”还是“多结构 + 少操作”

若节点类型老在变，访问者模式不一定合算。

2. 在 C 中可以用“遍历函数 + 回调 / ops 表”表达核心思想

不一定非要追求类 OO 形式。

3. 让节点结构只保存结构相关数据

不要把打印、校验、导出等逻辑都塞回节点定义里。

4. 访问过程中要注意栈深度与资源限制

若结构是树或递归结构，在 MCU 上要特别关注：

- 递归深度
 - 临时缓冲区
 - 访问过程中的锁与上下文
-

九、典型应用

1. Devicetree / 配置树遍历

Zephyr 提供了一组 devicetree 的“for-each”宏，例如 `DT_FOREACH_CHILD()`、`DT_FOREACH_NODE()`，允许在不改节点描述本身的前提下，对节点集施加不同操作。它不一定是教科书式双分派，但非常符合访问者模式“结构稳定、操作外置”的核心思想。[□cite□turn206503search2□turn206503search3□turn206503search7□](#)

2. 配置校验 / 导出 / 统计

同一棵配置树，可能既要打印、又要做合法性检查、又要生成默认值表，这些操作都适合做成外置访问逻辑。

3. 协议树与脚本树处理

例如调试器、配置编译器、代码生成工具，对同一组节点执行多种分析与导出操作，这是访问者模式最适合发力的地方。

总结

访问者模式在嵌入式 C 里最值得抓住的一点是：当节点结构相对稳定、而新操作还会不断增加时，把这些操作放到结构外部去扩展。